

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 PROJE RAPORU

Veritabanı Performans Optimizasyonu ve İzleme

Veri Temizleme ve ETL Süreçleri Tasarımı

Veritabanı Yükseltme ve Sürüm Yönetimi

Muhammed Eren Köseoğlu

22290604

Github: <https://github.com/Mamyy25/BLM4522>

Proje Videosu: https://drive.google.com/file/d/1WXsL00UHfKS-5ZmVMtVwV_VwG0-DQ2w7/view?usp=drive_link

İçindekiler Tablosu

1. Projelerin Genel Özeti

2. Veritabanı Performans Optimizasyonu ve İzleme

2.1 Kapsam ve Amaç

2.2 Kazanımlar ve Öğretiler

2.3 Kod Önizlemesi

2.3.1 Bölüm 1 – Veritabanı İzleme (pg_stat_statements)

2.3.2 Bölüm 2 – İndeks Yönetimi ve Sorgu İyileştirme

2.3.3 Bölüm 3 – Veri Yöneticisi Roller

3. Veri Temizleme ve ETL Süreçleri Tasarımı

3.1 Kapsam ve Amaç

3.2 Kazanımlar ve Öğretiler

3.3 Kod Önizlemesi

3.3.1 Bölüm 1 – Staging Tablosu ve Kirli Veri

3.3.2 Bölüm 2 – Veri Temizleme ve Dönüştürme

3.3.3 Bölüm 3 – Veri Yükleme (ETL Load)

3.3.4 Bölüm 4 – Veri Kalitesi Raporu

4. Veritabanı Yükseltme ve Sürüm Yönetimi

4.1 Kapsam ve Amaç

4.2 Kazanımlar ve Öğretiler

4.3 Kod Önizlemesi

4.3.1 Bölüm 1 – SchemaChangeLog Tablosu

4.3.2 Bölüm 2 – Event Trigger Fonksiyonu

4.3.3 Bölüm 3 – Yükseltme ve Geri Dönüş Senaryosu

4.3.4 Bölüm 4 – Sürüm Yönetimi Raporu

5. Sonuç

1. Projelerin Genel Özeti

Bu çalışma kapsamında, PostgreSQL veritabanı yönetim sistemi üzerinde üç farklı proje geliştirilmiştir: Performans Optimizasyonu ve İzleme, Veri Temizleme ve ETL Süreçleri Tasarımı ile Veritabanı Yükseltme ve Sürüm Yönetimi. Söz konusu projelerin temel amacı; teorik veritabanı yönetim konseptlerini gerçek hayat senaryolarıyla pratiğe dökmektir. Tüm projeler TechMarketDB adlı örnek veritabanı üzerinde uygulanmış olup PostgreSQL'e özgü araç ve sözdizimi kullanılmıştır.

2. Veritabanı Performans Optimizasyonu ve İzleme

2.1 Kapsam ve Amaç

Bu proje, PostgreSQL veritabanındaki darboğazları tespit etmek ve sorgu performansını artırmak amacıyla hazırlanmıştır. Çalışma kapsamında; sunucu genelinde en fazla kaynak tüketen sorgular pg_stat_statements eklentisiyle raporlanmış, Siparisler tablosundaki MusteriID kolonuna B-Tree indeksi oluşturularak sorgu planlarının Seq Scan'dan Index Scan'a geçişi gözlemlenmiştir. Ayrıca performans izleme görevini üstlenecek özel bir kullanıcı rolü tanımlanmıştır.

2.2 Kazanımlar ve Öğretiler

- Veritabanı İzleme: pg_stat_statements eklentisi ile sistemdeki en maliyetli sorgular tespit edilerek analiz edilmiştir.
- İndeks Etkisi: Aynı sorgunun indeksiz (Seq Scan) ve indeksli (Index Only Scan) çalışma planları karşılaştırılarak indeksin sorgu süresine olan etkisi pratik olarak gözlemlenmiştir.
- Atıl İndeks Tespiti: pg_stat_user_indexes görünümü ile kullanılmayan (user_scans = 0) indeksler belirlenmiş; bu indekslerin yalnızca yazma maliyeti yarattığı ve kaldırılması gerektiği öğrenilmiştir.
- Rol Tabanlı İzleme Yetkisi: Verileri değiştirme yetkisi olmayan, yalnızca sistem kaynaklarını okuyabilen özel bir izleme kullanıcısı (perfmonitoruser) oluşturulmuştur.

2.3 Kod Önizlemesi

2.3.1 Bölüm 1 – Veritabanı İzleme (pg_stat_statements)

Ne Yapıldı: Sunucudaki tüm sorguların istatistiklerini toplayan pg_stat_statements eklentisi etkinleştirildi. Ardından toplam yürütme süresine göre sıralanmış ilk 5 en maliyetli sorgu listelendi.

Kullanılan Kod:

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT
    calls AS execution_count,
    total_exec_time AS total_worker_time,
    mean_exec_time AS avg_worker_time,
    query AS query_text
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 5;
```

Query

Query History

1

-- =====

2

-- VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME (PROJE 1 - POSTGRESQL)

3

-- =====

4

5

-- 1. Veritabanı İzleme: pg_stat_statements eklentisi ile En Çok Kaynak Tüketen Sorguları Bulma

6

CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

7

8

-- Sistemde en fazla toplam yürütme zamanı (total_exec_time) harcayan ilk 5 sorgu:

9

SELECT

10

calls AS execution_count,

11

total_exec_time AS total_worker_time,

12

mean_exec_time AS avg_worker_time,

13

query AS query_text

14

FROM pg_stat_statements

15

ORDER BY total_exec_time DESC

16

LIMIT 5;

Data Output

Messages

Notifications

2.3.2 Bölüm 2 – İndeks Yönetimi ve Sorgu İyileştirme

Ne Yapıldı: Anlamlı bir performans karşılaştırması yapabilmek için Siparisler tablosuna 1000 satır test verisi eklendi. İndeks olmadan EXPLAIN ANALYZE çalıştırılarak "Seq Scan" gözlemlendi. Ardından MusteriID kolonuna Tutar ve SiparisTarihi sütunlarını kapsayan bir B-Tree indeksi oluşturuldu. Aynı sorgu tekrar çalıştırıldığında sorgu planının "Index Only Scan" moduna geçtiği doğrulandı.

Kullanılan Kod:

```
-- 1000 satır test verisi ekleme
INSERT INTO Siparisler (MusteriID, SiparisTarihi, Tutar)
SELECT
    (random() * 3 + 1)::INT,
    NOW() - (random() * 365 || ' days')::INTERVAL,
    round((random() * 5000 + 100)::NUMERIC, 2)
FROM generate_series(1, 1000);

-- İndeksiz sorgu planı (Seq Scan)
DROP INDEX IF EXISTS ix_siparisler_musteriid;
EXPLAIN ANALYZE
SELECT MusteriID, Tutar, SiparisTarihi
FROM Siparisler WHERE MusteriID = 1;

-- B-Tree İndeks oluşturma
CREATE INDEX ix_siparisler_musteriid
    ON Siparisler (MusteriID) INCLUDE (Tutar, SiparisTarihi);

-- İndeksli sorgu planı (Index Only Scan)
EXPLAIN ANALYZE
SELECT MusteriID, Tutar, SiparisTarihi
```

```
FROM Siparisler WHERE MusteriID = 1;
```

```
-- Atil indeks tespiti
SELECT relname AS table_name, indexrelname AS index_name,
       idx_scan AS user_scans
FROM pg_stat_user_indexes
WHERE relname = 'siparisler';
```

Query

Query History

```
26      round((random() * 5000 + 100)::NUMERIC, 2)
27  FROM generate_series(1, 1000);
28
29  -- Tablo istatistiklerini güncelle
30  ANALYZE Siparisler;
31
32  -- ADIM 2b: İndeks OLMADAN sorgu → "Seq Scan"
33  DROP INDEX IF EXISTS ix_siparisler_musteriid;
34
35  EXPLAIN ANALYZE
36  SELECT MusteriID, Tutar, SiparisTarihi
37  FROM Siparisler
38  WHERE MusteriID = 1;
```

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

🗑️

📦

⬇️

📈

SQL

S

QUERY PLAN

text

🔒

1	Seq Scan on siparisler (cost=0.00..19.56 rows=179 width=18) (actual time=0.011..0.061 rows=179.00 loop...
2	Filter: (musteriid = 1)
3	Rows Removed by Filter: 826
4	Buffers: shared hit=7
5	Planning:
6	Buffers: shared hit=20
7	Planning Time: 0.866 ms
8	Execution Time: 0.073 ms

Total rows: 8

Query complete 00:00:00.044

Query
Query History

```

36 SELECT MusteriID, Tutar, SiparisTarihi
37 FROM Siparisler
38 WHERE MusteriID = 1;
39
40 -- ADIM 2c: B-Tree Index
41 CREATE INDEX ix_siparisler_musteriid ON Siparisler (MusteriID) INCLUDE (Tutar, SiparisTarihi);
42
43 -- ADIM 2d: Aynı sorguyu İNDEKSİLİ çalıştır
44 EXPLAIN ANALYZE
45 SELECT MusteriID, Tutar, SiparisTarihi
46 FROM Siparisler
47 WHERE MusteriID = 1;
48
49 -- ADIM 2e: İndeks kullanım istatistikleri

```

Data Output
Messages
Notifications

Showing rows: 1 to 12

	QUERY PLAN	
	text	
1	Bitmap Heap Scan on siparisler (cost=9.66..18.90 rows=179 width=18) (actual time=0.046..0.072 rows=179.00 loops=1)	
2	Recheck Cond: (musteriid = 1)	
3	Heap Blocks: exact=7	
4	Buffers: shared hit=7 read=2	
5	-> Bitmap Index Scan on ix_siparisler_musteriid (cost=0.00..9.62 rows=179 width=0) (actual time=0.036..0.036 rows=179.00 loops=1)	
6	Index Cond: (musteriid = 1)	
7	Index Searches: 1	
8	Buffers: shared read=2	
9	Planning:	
Total rows: 12	Query complete 00:00:00.066	

2.3.3 Bölüm 3 – Veri Yöneticisi Roller

Ne Yapıldı: Performans izleme işlemleri için özel bir kullanıcı (perfmonitoruser) oluşturuldu ve PostgreSQL'in yerleşik okuma-izleme rolü olan pg_monitor bu kullanıcıya atandı. Bu sayede kullanıcı verilere müdahale edemeden sistem durumunu gözlemleyebilmektedir.

Kullanılan Kod:

```

DO $$ BEGIN
    IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'perfmonitoruser')
    THEN
        CREATE ROLE perfmonitoruser WITH LOGIN PASSWORD 'PerfPassword123!';
    END IF;
END $$;

GRANT pg_monitor TO perfmonitoruser;

```

3. Veri Temizleme ve ETL Süreçleri Tasarımı

3.1 Kapsam ve Amaç

Bu proje, dış sistemlerden gelen standart dışı ve hatalı verilerin (negatif tutar, eksik tarih, geçersiz müşteri ID'si vb.) doğrudan ana tabloya alınmadan önce bir geçici (Staging) tablo üzerinden ayıklanması, temizlenmesi ve dönüştürülmesi sürecini kapsamaktadır. ETL (Extract, Transform, Load) döngüsünün tüm adımları PostgreSQL ortamında uygulanmış; işlem sonucunda veri kalitesi bir raporla doğrulanmıştır.

3.2 Kazanımlar ve Öğretiler

- Staging Katmanı: Ham verinin doğrudan üretim tablosuna yazılmadan önce ayrı bir geçici tabloda tutulmasının önemi kavranmıştır.
- Kural Tabanlı Temizleme: Negatif tutar, eksik tarih ve geçersiz yabancı anahtar gibi farklı hata tiplerinin ayrı kurallarla ele alınması pratiği edinilmiştir.
- Dönüştürme (Transform): Eksik tarihlerin CURRENT_TIMESTAMP ile otomatik olarak doldurulması gibi düzeltilebilir hataların nasıl işleneceği öğrenilmiştir.
- Veri Kalitesi Raporlama: ETL sürecinin sonunda Durum sütununa göre gruplanan kayıt sayıları ve toplam tutarlar raporlanarak sürecin izlenebilirliği sağlanmıştır.

3.3 Kod Önizlemesi

3.3.1 Bölüm 1 – Staging Tablosu ve Kirli Veri

Ne Yapıldı: Dış sistemden gelen verilerin ilk yükleneceği Siparisler_Staging tablosu oluşturuldu. İçine bilerek hatalı veriler (eksik tarih, negatif tutar, olmayan müşteri ID'si) eklenerek gerçek bir kirli veri senaryosu simüle edildi.

Kullanılan Kod:

```
DROP TABLE IF EXISTS Siparisler_Staging;
CREATE TABLE Siparisler_Staging (
    TempID          SERIAL PRIMARY KEY,
    MusteriID       INT,
    SiparisTarihi    TIMESTAMP,
    Tutar            DECIMAL(10,2),
    Durum            VARCHAR(50) DEFAULT 'Bekliyor'
);

INSERT INTO Siparisler_Staging (MusteriID, SiparisTarihi, Tutar) VALUES
(1, '2023-11-01 10:00:00', 1500.00), -- Geçerli
(2, NULL, 500.00), -- Hata: Tarih yok
(3, '2023-11-02 12:00:00', -100.00), -- Hata: Negatif tutar
(99, '2023-11-03 09:00:00', 250.00), -- Hata: Olmayan müşteri
(4, '2023-11-04 15:00:00', 3000.00); -- Geçerli
```

3.3.2 Bölüm 2 – Veri Temizleme ve Dönüştürme

Ne Yapıldı: Staging tablosundaki veriler üç kurala göre işlendi: negatif tutarlar reddedildi, tarihi boş olan kayıtlara mevcut tarih-saat atandı, Musteriler tablosunda karşılığı olmayan müşteri ID'leri geçersiz olarak işaretlendi.

Kullanılan Kod:

```
-- Kural 1: Negatif tutarlar reddedilir
UPDATE Siparisler_Staging
SET Durum = 'Reddedildi: Negatif Tutar'
WHERE Tutar <= 0;

-- Kural 2: Tarihi olmayanlara bugünün tarihi atanır (Dönüştürme)
UPDATE Siparisler_Staging
SET SiparisTarihi = CURRENT_TIMESTAMP,
    Durum = 'Düzeltilildi: Tarih Eklendi'
WHERE SiparisTarihi IS NULL AND Durum = 'Bekliyor';

-- Kural 3: Geçersiz müşteri ID'leri reddedilir
UPDATE Siparisler_Staging
SET Durum = 'Reddedildi: Gecersiz Musteri'
WHERE MusteriID NOT IN (SELECT MusteriID FROM Musteriler)
    AND Durum = 'Bekliyor';
```

Query	Query History
26	
27	-- Kural 1: Tutarı negatif olan hatalı kayıtları 'Reddedildi' olarak işaretle
28	UPDATE Siparisler_Staging
29	SET Durum = 'Reddedildi: Negatif Tutar'
30	WHERE Tutar <= 0;
31	
32	-- Kural 2: Sipariş tarihi olmayanlara bugünün tarihini atayarak dönüştür
33	UPDATE Siparisler_Staging
34	SET SiparisTarihi = CURRENT_TIMESTAMP, Durum = 'Düzeltilildi: Tarih Eklendi'
35	WHERE SiparisTarihi IS NULL AND Durum = 'Bekliyor';
36	
37	-- Kural 3: Müşteriler tablosunda olmayan MusteriID'ye sahip kayıtları reddet
38	UPDATE Siparisler_Staging
39	SET Durum = 'Reddedildi: Gecersiz Musteri'
40	WHERE MusteriID NOT IN (SELECT MusteriID FROM Musteriler) AND Durum = 'Bekliyor';
41	
42	SELECT TempID, MusteriID, SiparisTarihi, Tutar, Durum FROM Siparisler_Staging;

tempid	musteriid	siparistarihi	tutar	durum
[PK] integer	integer	timestamp without time zone	numeric (10,2)	character varying (50)
1	1	2023-11-01 10:00:00	1500.00	Bekliyor
2	5	2023-11-04 15:00:00	3000.00	Bekliyor
3	3	2023-11-02 12:00:00	-100.00	Reddedildi: Negatif Tutar
4	2	2026-06-08 15:35:32.552115	500.00	Düzeltilildi: Tarih Eklendi
5	4	2023-11-03 09:00:00	250.00	Reddedildi: Gecersiz Must...

Total rows: 5 Query complete 00:00:00.201

3.3.3 Bölüm 3 – Veri Yükleme (ETL Load)

Ne Yapıldı: Staging tablosunda "Bekliyor" veya "Düzeltilildi" durumundaki temiz ve geçerli kayıtlar ana Siparisler tablosuna aktarıldı. Yüklenen kayıtların durumu "Başarıyla Yüklendi" olarak güncellendi ve ETL döngüsü tamamlandı.

Kullanılan Kod:

```
-- Temiz verileri ana tabloya aktar
INSERT INTO Siparisler (MusteriID, SiparisTarihi, Tutar)
SELECT MusteriID, SiparisTarihi, Tutar
FROM Siparisler_Staging
WHERE Durum = 'Bekliyor' OR Durum LIKE 'Düzeltilildi%';
```

```
-- Yüklenen kayıtları işaretle
UPDATE Siparisler_Staging
SET Durum = 'Başarıyla Yüklendi'
WHERE Durum = 'Bekliyor' OR Durum LIKE 'Düzeltildi%';
```

Query Query History

```

42 SELECT TempID, MusteriID, SiparisTarihi, Tutar, Durum FROM Siparisler_Staging;
43
44 -- 3. Veri Yükleme (Data Loading)
45 -- Sadece temiz ve geçerli verileri ana 'Siparisler' tablosuna aktarma
46 INSERT INTO Siparisler (MusteriID, SiparisTarihi, Tutar)
47 SELECT MusteriID, SiparisTarihi, Tutar
48 FROM Siparisler_Staging
49 WHERE Durum = 'Bekliyor' OR Durum LIKE 'Düzeltildi%';
50
51 -- Yüklenen kayıtları Staging tablosunda işaretle
52 UPDATE Siparisler_Staging
53 SET Durum = 'Başarıyla Yüklendi'
54 WHERE Durum = 'Bekliyor' OR Durum LIKE 'Düzeltildi%';
55
56
57 -- 4. Veri Kalitesi Raporları
58 -- İşlem sonucu verilerin durumunu raporlama

```

Data Output Messages Notifications

Showing rows: 1 to 5

Page No: 1

	tempid [PK] integer	musteriid integer	siparistarihi timestamp without time zone	tutar numeric (10,2)	durum character varying (50)
1	3	3	2023-11-02 12:00:00	-100.00	Reddedildi: Negatif Tutar
2	4	99	2023-11-03 09:00:00	250.00	Reddedildi: Gecersiz Must...
3	1	1	2023-11-01 10:00:00	1500.00	Başarıyla Yüklendi
4	5	4	2023-11-04 15:00:00	3000.00	Başarıyla Yüklendi
5	2	2	2026-06-08 15:35:32.552115	500.00	Başarıyla Yüklendi

Total rows: 5

Query complete 00:00:00.069

3.3.4 Bölüm 4 – Veri Kalitesi Raporu

Ne Yapıldı: ETL sürecinin tamamlanmasının ardından Staging tablosundaki tüm kayıtların durum bazında sayıları ve toplam tutarları raporlandı. Bu rapor sayesinde kaç kaydın başarıyla yüklendiği, kaçının hangi nedenle reddedildiği açıkça görülebildi.

Kullanılan Kod:

```

SELECT
    Durum,
    COUNT(*) AS KayitSayisi,
    SUM(Tutar) AS ToplamTutar
FROM Siparisler_Staging
GROUP BY Durum;

```

4. Veritabanı Yükseltme ve Sürüm Yönetimi

4.1 Kapsam ve Amaç

Bu proje, PostgreSQL veritabanındaki şema değişikliklerinin (CREATE, ALTER, DROP) otomatik olarak kayıt altına alınmasını sağlayan bir sürüm yönetimi mekanizmasının tasarlanmasını ve uygulanmasını kapsamaktadır. Çalışma kapsamında bir SchemaChangeLog tablosu oluşturulmuş, PL/pgSQL ile yazılan Event Trigger fonksiyonları bu tabloya DDL işlemlerini otomatik olarak kaydetmiştir. Gerçek bir sürüm geçiş senaryosu simüle edilerek yeni modül ekleme, değiştirme ve geri alma adımlarının tamamının log tablosuna yansıdığı doğrulanmıştır.

4.2 Kazanımlar ve Öğretiler

- Event Trigger Mekanizması: PostgreSQL'e özgü EVENT TRIGGER yapısının MSSQL'deki DDL Trigger'ların karşılığı olduğu ve ddl_command_end ile sql_drop gibi farklı olayları yakaladığı öğrenilmiştir.
- PL/pgSQL Fonksiyon Yazımı: pg_event_trigger_ddl_commands() ve pg_event_trigger_dropped_objects() sistem fonksiyonlarının olay bilgilerini nasıl döndürdüğü ve döngüyle nasıl işlendiği pratikte görülmüştür.
- Geri Dönüş (Rollback) Planlaması: Yeni bir sürümde başarısız olan modülün tablosunun DROP ile kaldırılmasının, üretim ortamında sistemi önceki sürüme döndürmenin pratik bir yolu olduğu kavranmıştır.
- İzlenebilirlik: "Kim, Neyi, Ne Zaman değiştirdi?" sorusunun yanıtının SchemaChangeLog tablosu aracılığıyla her zaman sorgulanabilir olması, kurumsal veritabanı yönetiminde kritik bir gereklilik olarak görülmüştür.

4.3 Kod Önizlemesi

4.3.1 Bölüm 1 – SchemaChangeLog Tablosu

Ne Yapıldı: Veritabanı yapısında gerçekleşen DDL değişikliklerini (işlem türü, nesne adı, kullanıcı, tarih) kayıt altına alacak SchemaChangeLog tablosu oluşturuldu.

Kullanılan Kod:

```
DROP TABLE IF EXISTS SchemaChangeLog;
CREATE TABLE SchemaChangeLog (
    LogID          SERIAL PRIMARY KEY,
    EventDate      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    EventType      VARCHAR(100),
    ObjectType     VARCHAR(100),
    ObjectName     VARCHAR(255),
    LoginName     VARCHAR(255)
);
```

4.3.2 Bölüm 2 – Event Trigger Fonksiyonu

Ne Yapıldı: CREATE ve ALTER işlemlerini yakalayan bir PL/pgSQL fonksiyonu (trg_log_schema_changes) ile DROP işlemlerini yakalayan ayrı bir fonksiyon (trg_log_drop_changes) yazıldı. Her iki fonksiyon da ilgili Event Trigger'lara bağlanarak tüm DDL işlemlerinin SchemaChangeLog tablosuna otomatik kaydedilmesi sağlandı.

Kullanılan Kod:

```
-- CREATE / ALTER işlemlerini yakalayan fonksiyon
CREATE OR REPLACE FUNCTION trg_log_schema_changes()
RETURNS event_trigger AS $$
DECLARE r RECORD;
BEGIN
    FOR r IN SELECT * FROM pg_event_trigger_ddl_commands() LOOP
        INSERT INTO SchemaChangeLog (EventType, ObjectType, ObjectName,
        LoginName)
            VALUES (TG_TAG, r.object_type, r.object_identity, current_user);
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- DROP işlemlerini yakalayan fonksiyon
CREATE OR REPLACE FUNCTION trg_log_drop_changes()
RETURNS event_trigger AS $$
DECLARE r RECORD;
BEGIN
    FOR r IN SELECT * FROM pg_event_trigger_dropped_objects() LOOP
        INSERT INTO SchemaChangeLog (EventType, ObjectType, ObjectName,
        LoginName)
            VALUES (TG_TAG, r.object_type, r.object_identity, current_user);
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Event Trigger'lar oluşturuluyor
DROP EVENT TRIGGER IF EXISTS track_schema_changes;
DROP EVENT TRIGGER IF EXISTS track_drop_changes;

CREATE EVENT TRIGGER track_schema_changes ON ddl_command_end
    EXECUTE FUNCTION trg_log_schema_changes();

CREATE EVENT TRIGGER track_drop_changes ON sql_drop
    EXECUTE FUNCTION trg_log_drop_changes();
```

Query

Query History

Scratch Pad

15

16

SELECT * FROM SchemaChangeLog;

Data Output

Messages

Notifications

<

4.3.3 Bölüm 3 – Yükseltme ve Geri Dönüş Senaryosu

Ne Yapıldı: Sisteme yeni bir modül eklendiği düşünülerek YeniModul_TestTablosu oluşturuldu (CREATE). Ardından bu tabloya yeni bir sütun eklendi (ALTER). Son olarak modülün başarısız olduğu varsayılarak tablo kaldırıldı (DROP – Geri Dönüş/Rollback). Bu üç adımın tamamı Event Trigger tarafından otomatik olarak kayıt altına alındı.

Kullanılan Kod:

```
-- Yeni sürüm: Modül tablosu ekleniyor (CREATE)
CREATE TABLE YeniModul_TestTablosu (
    TestID INT PRIMARY KEY,
    TestAdi VARCHAR(50)
);

-- Sürüm güncellemesi: Tabloya yeni kolon ekleniyor (ALTER)
ALTER TABLE YeniModul_TestTablosu
    ADD COLUMN Aciklama VARCHAR(200);

-- Geri Dönüş Planı: Modül başarısız → tablo kaldırılıyor (DROP)
DROP TABLE YeniModul_TestTablosu;
```

4.3.4 Bölüm 4 – Sürüm Yönetimi Raporu

Ne Yapıldı: Yukarıdaki üç DDL işleminin (CREATE TABLE, ALTER TABLE, DROP TABLE) Event Trigger tarafından başarıyla yakalandığı SchemaChangeLog tablosu sorgulanarak doğrulandı. Raporda her işlemin türü, nesne adı ve gerçekleştiği zaman bilgisi görüntülendi.

Kullanılan Kod:

```
SELECT LogID, EventDate, EventType, ObjectType, ObjectName, LoginName
FROM SchemaChangeLog
ORDER BY EventDate DESC;
```

5. Sonuç

Bu final projesi kapsamında gerçekleştirilen üç çalışma, PostgreSQL veritabanı yönetiminin farklı boyutlarını ele almıştır. Performans Optimizasyonu projesinde indekslemenin sorgu maliyetleri üzerindeki somut etkisi gözlemlenmiş ve sistem izleme yetki yönetimi pratikte uygulanmıştır. ETL projesinde ham verinin üretim ortamına aktarılmadan önce bir Staging katmanında nasıl temizlenip dönüştürülmesi gerektiği öğrenilmiştir. Sürüm Yönetimi projesinde ise PostgreSQL'in Event Trigger mekanizması aracılığıyla şema değişikliklerinin otomatik olarak loglanması sağlanmış ve kurumsal veritabanı ortamlarında izlenebilirliğin önemi kavranmıştır. Tüm bu projeler, veritabanı yönetiminde teorik bilginin pratiğe aktarılmasında önemli katkılar sağlamıştır.